MyBatis 面试题速记通关版 _ 面试刷题 mianshiya.com

> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*

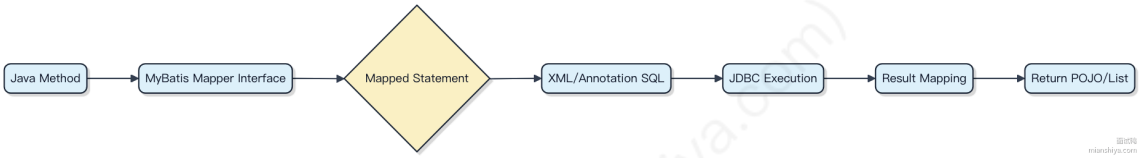
MyBatis 的优点？

MyBatis 把 SQL 和代码解耦，写在 XML 或注解里，改 SQL 不用动 Java 代码，维护起来方便多了。尤其适合复杂查询、多表关联这种需要精细控制 SQL 的场景，不像 JPA 那样生成的 SQL 有时不可控。

你想要什么格式的返回结果，直接在映射文件里定义 ResultMap 就行，对象字段和数据库列的映射关系一目了然，处理字段名不一致、嵌套对象都挺顺手。

1）SQL 交给你自己写，性能好调，索引怎么走、要不要加 hint 都能控制
2）不像全自动 ORM 那样有学习成本，会 SQL 就能上手
3）可以复用 SQL 片段，比如几个查询都要用到 `deleted = 0`，写个 `<sql>` 块引用就行

但脏活累活也得你干。一级缓存默认开启，基于 SqlSession，同一线程查两次同 SQL 会走缓存；二级缓存跨 SqlSession，但粒度粗，容易脏读，实际项目里很多人直接关掉，用 Redis 自己管。



拿电商系统举例，订单列表要连用户、商品、物流三张表，SQL 复杂还带动态过滤条件，用 MyBatis 写 `<where>` 和 `<if>` 拼接，比拼对象再转 SQL 痛快多了。不过批量插入十万条数据这种场景，可能还是 JDBC 直接搞或者用 ShardingSphere 处理更稳。

MyBatis 自带的连接池有了解过吗？

MyBatis 自带的连接池叫 PooledDataSource，它不是拿来替换 Druid 或 HikariCP 的，更多是保证基础可用性。你不用额外依赖第三方库，也能跑起来。

连接池的核心是维护两个状态：空闲连接和活跃连接。PooledDataSource 用一个 idleConnections 队列存空闲连接，activeConnections 存正在用的。获取连接时，先看队列里有没有现成的，有就直接拿，没有就新建，但总数不能超 poolMaximumActiveConnections，默认是 10。

归还连接的时候，不是真关掉，而是清空状态，塞回 idleConnections。如果空闲数超了上限，或者连接失效了，就彻底关闭。

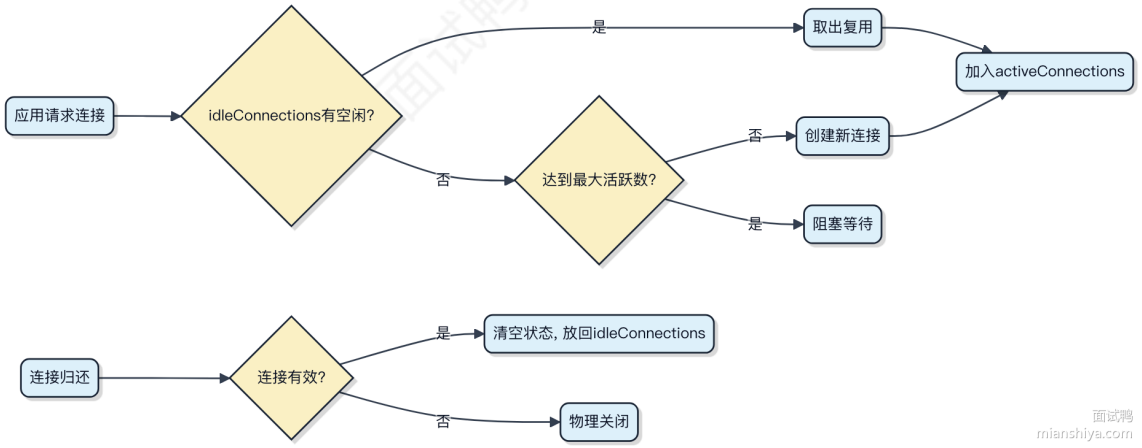
1）连接分配时会做有效性检查，比如检测是否已关闭、网络是否断开
2）支持坏连接回收，避免长期占用资源

3）有简单的阻塞等待机制，拿不到连接的线程会 wait，其他线程归还后 notify

代码层面，你配置数据源时写 type="POOLED" 就启用了：

```
<dataSource type="POOLED">
  <property name="driver" value="${driver}"/>
  <property name="url" value="${url}"/>
  <property name="username" value="${username}"/>
  <property name="password" value="${password}"/>
</dataSource>
```

实际生产基本都用 **HikariCP** 或 Druid，性能更好，监控更全。MyBatis 这个适合小项目或学习阶段，省事，但压根不经过复杂场景考验。你要做高并发系统，肯定得换。



使用 MyBatis 的 mapper 接口调用时有哪些要求？

接口方法名必须和 Mapper XML 中的 statement id 一致，参数类型也要对应。比如 XML 里写的是 `selectUserById`，那接口方法就得叫 `selectUserById`，不然找不到。

1） 接口全限定名要和 XML 文件的 namespace 完全匹配。这是 MyBatis 绑定的依据，差一个字母都找不到。

2） 方法的返回值类型得和 XML 里 resultType 或 resultMap 能映射上。如果数据库返回 3 个字段，但实体类没有无参构造或者属性不匹配，直接抛异常。

3） 参数只能有一个，如果你有多个参数，得用 `@Param` 注解命名，否则 XML 里没法通过名字引用。比如 `int updateUser(@Param("id") Long id, @Param("name") String name)`，XML 才能用 `#{id}` 和 `#{name}`。

4） XML 文件必须和接口在同一个包下，且同名。Spring Boot 默认扫描 classpath*:/**/*.xml，但前提是路径对。Maven 还得注意别把 XML 过滤掉了。

代码示例：

```
public interface UserMapper {
    User selectUserById(Long id);
}
```

对应 XML：

```
<select id="selectUserById" resultType="User">
    SELECT * FROM user WHERE id = #{id}
</select>
```

实际项目里，像 **MyBatis-Plus** 就是靠动态代理 + 这套规则自动生成 SQL，你不用写 XML。但底层还是遵守这些约定。

Mybatis 如何实现一对一、一对多的关联查询？

一对一和一对多查询，关键在于 `resultMap` 的嵌套映射能力。Mybatis 不像 Hibernate 那样自动维护对象关系，它靠的是手动配置字段与 Java 对象属性的对应规则。

1） 一对一用 `<association>` 标签。比如订单 Order 关联用户 User，Order 类里有个 user 属性。你得定义一个 `resultMap` ，把 order 的字段映射完，再通过 association 指向另一个 User 的 resultMap 或直接指定 type。

```
<resultMap id="OrderMap" type="Order">
    <id property="id" column="order_id"/>
    <association property="user" javaType="User">
        <id property="id" column="user_id"/>
        <result property="name" column="user_name"/>
    </association>
</resultMap>
```

2） 一对多用 `<collection>` 标签。典型场景是部门 Department 查员工列表 List。collection 的 ofType 指明集合泛型类型，可以嵌套 resultMap。

```
<resultMap id="DeptMap" type="Department">
    <id property="id" column="dept_id"/>
    <collection property="employees" ofType="Employee">
        <id property="id" column="emp_id"/>
        <result property="name" column="emp_name"/>
    </collection>
</resultMap>
```

SQL 层面一般用连表查询（JOIN），一条语句捞出所有数据，Mybatis 根据 result Map 自动组装成树形结构。虽然也能用分步查询（嵌套 select），但容易 N+1 性能问题，实际项目像 Spring Data JPA 才更倾向这种方式。

常见坑是 column 写错导致关联字段没映射上，或者 ofType 和 javaType 混用。调试时打开 Mybatis 日志，看最终生成的对象结构最直观。

MyBatis 中 #{} 和 ${} 的区别是什么？

`#{}` 和 `${}` 看似都是占位，但处理时机完全不同。

`#{}` 是预编译参数占位符，MyBatis 会把 SQL 中的 `#{}` 替换成 `?` ，然后通过 `PreparedStatement` 的 `set` 方法赋值。这个过程能有效防止 SQL 注入，因为参数内容不会参与 SQL 解析。比如：

```
<select id="getUser" resultType="User">
    SELECT * FROM user WHERE id = #{id}
</select>
```

最终执行的是 `SELECT * FROM user WHERE id = ?`，安全又高效。

`${}` 是字符串拼接，直接把变量值原样插入到 SQL 中，不经过预编译。这意味着如果传入恶意内容，就可能被当作 SQL 执行。比如：

```
ORDER BY ${columnName}
```

这里 `${columnName}` 如果是 `id; DROP TABLE user`，那后果就严重了。所以 `${}` 只能在无法使用预编译的场景下用，比如动态表名、动态排序字段，而且必须确保输入可控。

一般情况下，能用 `#{}` 就别碰 `${}`。非要用 `${}`，得自己做白名单校验，不能让用户随便输。

1） `#{}` 走预编译，安全，性能好
2） `${}` 是拼接，有注入风险，慎用
3） `${}` 适用场景：动态表名、动态排序、动态 in 条件（需转义）

MyBatis 写个 Xml 映射文件，再写个 DAO 接口就能执行，这个原理是什么？

MyBatis 能让接口和 XML 自动绑定，靠的是 JDK 动态代理和反射机制。你定义的 DAO 接口压根不会被实现，MyBatis 在运行时给它生成一个代理对象，所有方法调用都被拦截。

当你调用接口方法时，比如 `userMapper.selectById(1)`，代理会根据方法名 `selectById` 去找对应的 `<select>` 标签。这个匹配过程依赖命名空间——XML 的 namespace 必须是接口的全限定名，statementId 就是方法名。找到 SQL 后，再通过反射解析参数、执行语句、映射结果。

整个过程的关键在于 **SqlSession** 获取 Mapper 时触发了代理创建。比如你写 `sqlSession.getMapper(UserMapper.class)`，MyBatis 就知道你要操作哪个接口，然后返回一个代理实例。后续所有方法调用都走 SqlSession 的 CRUD 模板流程。

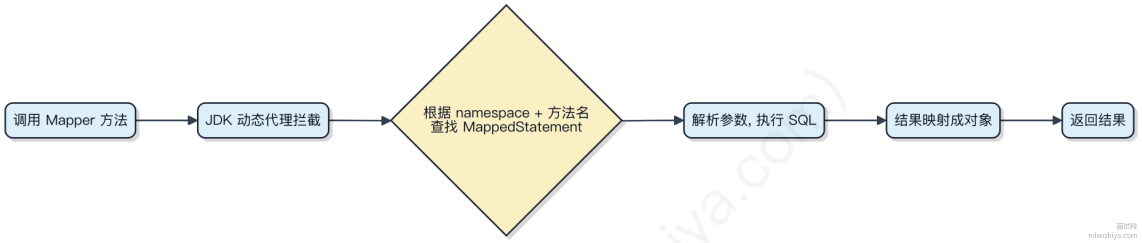
来看个精简代码示例：

```
// 接口
public interface UserMapper {
    User selectById(int id);
}
```

```
<!-- XML -->
<mapper namespace="UserMapper">
    <select id="selectById" resultType="User">
        SELECT * FROM user WHERE id = #{id}
    </select>
</mapper>
```

这里没有硬编码实现类，但能跑起来，就是因为代理在中间做了路由。如果命名对不上，比如 method 名和 id 不一致，启动时就会报错找不到 mapped statement。

这种设计让持久层逻辑彻底解耦，SQL 放 XML，调用走接口，Spring 集成后更是零实现类。



MyBatis 动态 sql 有什么用？执行原理？有哪些动态 sql？

动态 SQL 是为了解决拼接 SQL 字符串的痛点。你肯定遇到过根据条件决定是否添加 WHERE 子句的情况，以前靠手拼字符串，容易出错还难维护。

MyBatis 的动态 SQL 在解析阶段就把这些逻辑处理掉，最终生成一个干净的 SQL 语句交给数据库执行。整个过程发生在 SqlSource 构建时，通过 OGNL 表达式计算条件，生成对应的 BoundSql。

常见的标签有这几个：

1） `<if>` 最常用，条件成立才拼入 SQL
2） `<where>` 自动处理 AND/OR 开头问题，条件为空时不生成 WHERE
3） `<set>` 配合 update 使用，自动加 SET 关键字并去掉末尾逗号
4） `<trim>` 更底层的封装，可以自定义前后缀和前缀后缀去除规则
5） `<foreach>` 遍历集合，常用于 IN 查询或批量插入
6） `<choose>` 、 `<when>` 、 `<otherwise>` 类似 Java 的 switch-case

比如写个查询：

```
<select id="list" resultType="User">
  SELECT * FROM user
  <where>
    <if test="name != null">AND name = #{name}</if>
    <if test="age != null">AND age = #{age}</if>
  </where>
</select>
```

传 null 参数时，对应条件直接消失，不会影响其他部分。

注意 `<if>` 标签别乱堆，复杂场景用 `<trim>` 控制更精准。像 `IN` 查询这种必须用 `<foreach>` ，不然搞不定参数绑定。

MyBatis 与 Hibernate 有哪些不同？

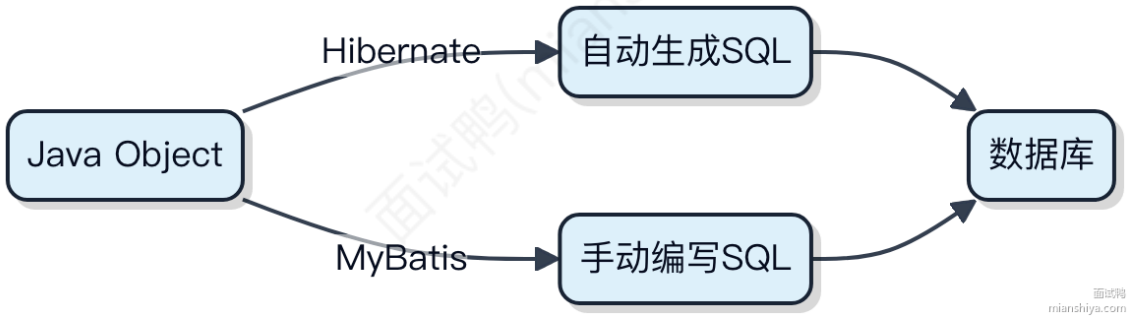
MyBatis 和 Hibernate 都是 Java 里处理数据库操作的 ORM 框架，但设计哲学差得挺远。

Hibernate 是全自动化的，你定义好实体类和映射关系，增删改查基本不用写 SQL，它自动生成并执行。比如用 `Session.save(entity)` 就能把对象存进数据库，连事务都帮你管好了。这种适合业务规则稳定、表结构规范的项目，像传统企业管理系统用 Hibernate 能省不少事。

MyBatis 则是半自动的，SQL 得自己写，通过 XML 或注解把参数和结果集跟 Java 对象做映射。比如一个带多表关联的查询，你可以精确控制 SQL 的每一部分，性能调优更方便。互联网公司搞高并发、复杂查询时，普遍选 MyBatis，毕竟对 SQL 的掌控力更强。

1）Hibernate 抹平了数据库差异，HQL 可以跨数据库运行，迁移成本低
2）MyBatis 直接暴露 SQL，灵活性高，但换数据库时可能要重写语句
3）Hibernate 有完整的缓存体系（一级、二级缓存），MyBatis 缓存功能相对简单

举个例子，做报表查询这种字段多、条件动态拼接的场景，Hibernate 很难搞定，SQL 不透明，拼起来费劲。MyBatis 直接写原生 SQL，配合动态标签 `<if>` 、 `<choose>` 处理条件，干净利落。



总的来说，追求开发效率、少写 SQL 用 Hibernate；需要精细控制 SQL、应对复杂场景就上 MyBatis。

MyBatis 是否支持延迟加载？如果支持，它的实现原理是什么？

MyBatis 支持延迟加载，但默认不开启，需要手动配置。

延迟加载的核心是**动态代理**。当你查一个带关联的对象时，比如查订单的同时查用户信息，如果开启了延迟加载，MyBatis 不会立刻去查用户表。它会返回一个 User 对象的代理，这个代理是用 Javassist 或 CGLIB 生成的。只有你真正调用这个代理对象的方法（比如 getUser().getName()）时，才会触发 SQL 去查数据库。

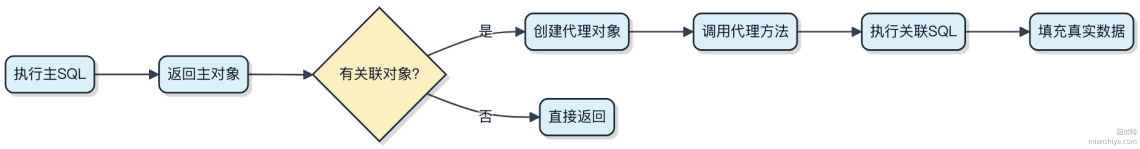
这种机制能减少不必要的关联查询，提升性能。比如列表页不需要详情数据时，就能避免 N+1 查询问题。典型场景就像查多个订单时不急着查每个订单的用户、商品信息。

要开启延迟加载，得在配置文件里设置：

```
<setting name="lazyLoadingEnabled" value="true"/>
<setting name="aggressiveLazyLoading" value="false"/>
```

第二个配置很关键，设为 false 才能保证只在调用具体方法时才加载，否则一访问对象就触发。

1）先查主数据，比如 `SELECT * FROM orders WHERE id = ?` 2）关联数据不立即查，返回代理对象 3）调用关联对象方法时，执行 `SELECT * FROM user WHERE id = ?`



说说 MyBatis 的缓存机制？

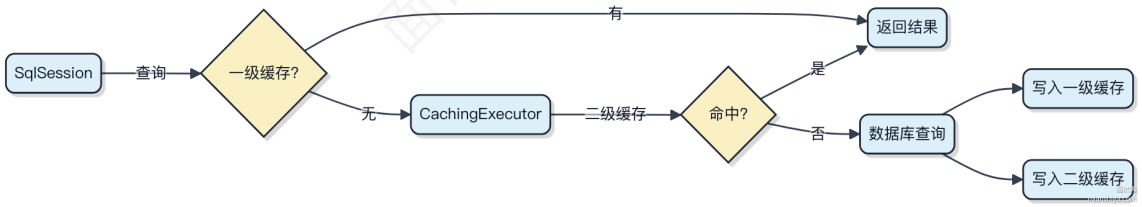
MyBatis 的缓存分两级，一级缓存默认开启，作用域是 SqlSession 级别。只要你用同一个 SqlSession 执行相同的 SQL，参数也一样，第二次查询直接从内存取结果，不用走数据库。

1）一级缓存底层其实就是一个 HashMap，数据结构是 key（MappedStatement ID + 查询参数）到缓存对象的映射。一旦执行 commit、update 或 close，缓存就清空。所以事务提交后，下个会话查的数据是新的。

2）二级缓存作用范围更广，跨 SqlSession，通常集成 Redis、Ehcache 来实现。需要手动开启，对应的 Mapper 类得实现 Serializable，配置文件里加上 `<cache/>` 。多个会话查相同 Mapper 和参数时，先看二级缓存有没有。

但要注意，二级缓存如果没设计好，可能出脏数据。比如 A 会话更新了数据还没提交，B 会话就从缓存读了旧值。一般在读多写少场景用，像商品信息这种，配合 TTL 控制失效时间。

代码上，关键点是 MappedStatement 的 cache 属性是否绑定缓存实例。每次执行查询，CachingExecutor 先问 cache 有没有，没有再委托给 BaseExecutor 去查数据库。



简单说，一级缓存防一次会话重复查，二级缓存扛住多个会话的重复请求。真并发高并发场景，还是建议用 Redis 自定义缓存策略，比 MyBatis 默认的更可控。

MyBatis 的缺点？

MyBatis 用得挺多，但真要挑毛病，几个地方确实让人头疼。

1）SQL 搞不定自动优化。你写啥它就执行啥，索引用没用、有没有慢查询，框架压根不干预。不像 JPA 那种能做一定抽象，MyBatis 把所有脏活累活都甩给开发者，稍微不留神，生产就出问题。

2）维护成本高到离谱。一张表改个字段，Mapper 接口、XML 文件、ResultMap、DAO 方法全得手动改一遍。项目大了以后，一个实体类变动牵出十几个文件要同步，搞不好就是空指针。Spring Data JPA 改个注解完事，这儿得手动翻三四个地方。

3）数据库迁移基本靠重写。不同数据库的 SQL 方言差异大，比如分页，MySQL 用 limit，Oracle 得套两层 rownum。想换个库？不好意思，所有 XML 里的 SQL 得挨个检查调整。虽然可以用 `<if>` 标签做数据库类型判断，但复杂 SQL 里这么干，可读性直接崩盘。

4）缓存设计太鸡肋。一级缓存默认开启但作用域窄，只在同一个 SqlSession 有效，Web 场景下基本没啥用。二级缓存跨 SqlSession 共享，但粒度粗，只要同一个 Mapper 的增删改就会清空整个 namespace 缓存，容易误伤。真要搞分布式缓存，还得自己集成 Redis，缓存一致性全靠手动维护。

代码示例，比如分页这种明显不兼容的：

```
<select id="list" resultType="User">
  <if test="database == 'mysql'">
    SELECT * FROM user LIMIT #{offset}, #{size}
  </if>
```

```
  <if test="database == 'oracle'">
    SELECT * FROM (SELECT ROWNUM rn, u.* FROM user u WHERE ROWNUM &lt;= #{end})
WHERE rn &gt; #{start}
  </if>
</select>
```

这种判断混在 SQL 里，后期看简直灾难。

简述 MyBatis 的插件运行原理，以及如何编写一个插件？

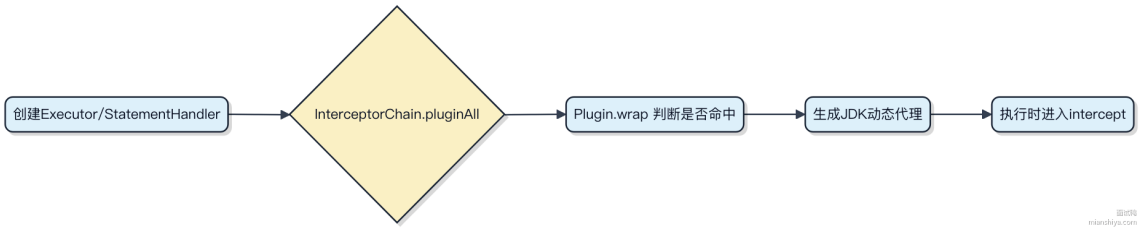
MyBatis 插件本质上是通过 **拦截器** 机制，对四大核心对象的方法调用进行增强。这四个对象分别是 Executor、StatementHandler、ParameterHandler 和 ResultSetHandler。插件不会侵入原有逻辑，而是利用动态代理，在关键方法执行时回调你定义的 intercept 方法。

1）要写一个插件，先实现 Interceptor 接口，重写 intercept 方法，里面写你的增强逻辑，比如打印 SQL 执行时间。
2）用 @Signature 注解声明你要拦截哪个类的哪个方法，目标明确，不能乱来。
3）最后在 mybatis-config.xml 里注册这个插件，或者用编程式配置加进去。

```
@Intercepts({
    @Signature(type = StatementHandler.class, method = "prepare", args =
{Connection.class, Integer.class})
})
public class SqlCostPlugin implements Interceptor {
    @Override
    public Object intercept(Invocation invocation) throws Throwable {
        long start = System.currentTimeMillis();
        try {
            return invocation.proceed();
        } finally {
            long cost = System.currentTimeMillis() - start;
            System.out.println("SQL 执行耗时: " + cost + "ms");
        }
    }
}
```

注意，多个插件会形成责任链，按注册顺序依次执行。代理逻辑由 Plugin.wrap 方法触发，它会判断当前对象是否匹配 @Signature 声明的目标，是的话就包一层代理。

整个过程其实就是在创建四大对象时，调用 interceptorChain.pluginAll() 把所有插件都套上去。比如 new Executor 时，会被各个插件的 plugin 方法处理，最终返回被代理后的实例。



JDBC 编程有哪些不足之处，MyBatis 是如何解决的？

JDBC 直接写在代码里，SQL 和 Java 逻辑紧耦合，改个字段就得重新编译，维护成本特别高。更头疼的是，结果集要手动映射到对象，每个 `ResultSet#getXxx` 都得写一遍，脏活累活全是程序员的。

1）SQL 耦合问题
原生 JDBC 把 SQL 拼在字符串里，不仅难读，还容易出错。MyBatis 用 XML 或注解把 SQL 抽出来，放在 `.xml` 文件里集中管理，DAO 接口只定义方法，SQL 独立可配，改查询不用动代码。

2）参数设置繁琐
JDBC 设置 `PreparedStatement` 参数，一个问号对应一个 `setXxx` ，顺序不能错。MyBatis 自动做参数映射，无论是单个参数还是 POJO，都能通过 `#{}` 对应上，压根不操心下标。

3）结果集手动封装
JDBC 必须一行行取值再 set 到对象里，比如 `user.setId(rs.getInt("id"))` ，重复代码一大堆。MyBatis 支持自动映射，查出来的列名直接对上实体类属性，甚至支持复杂嵌套关系， `<resultMap>` 搞定关联字段。

4）数据库方言差异
分页在 MySQL 用 `LIMIT` ，Oracle 得套多层 `ROWNUM` ，JDBC 写死 SQL 就没法跨库。MyBatis 可以结合动态标签 `<if>` 、 `<where>` 生成不同方言语句，换数据库时只需改 SQL 文件，代码不动。

简单说，MyBatis 不是 ORM 框架像 Hibernate 那样完全屏蔽 SQL，而是给 JDBC 套了层“智能外壳”，保留 **SQL 精细控制权**的同时，把重复劳动全自动化了。

Mybatis 都有哪些 Executor 执行器？它们之间的区别是什么？

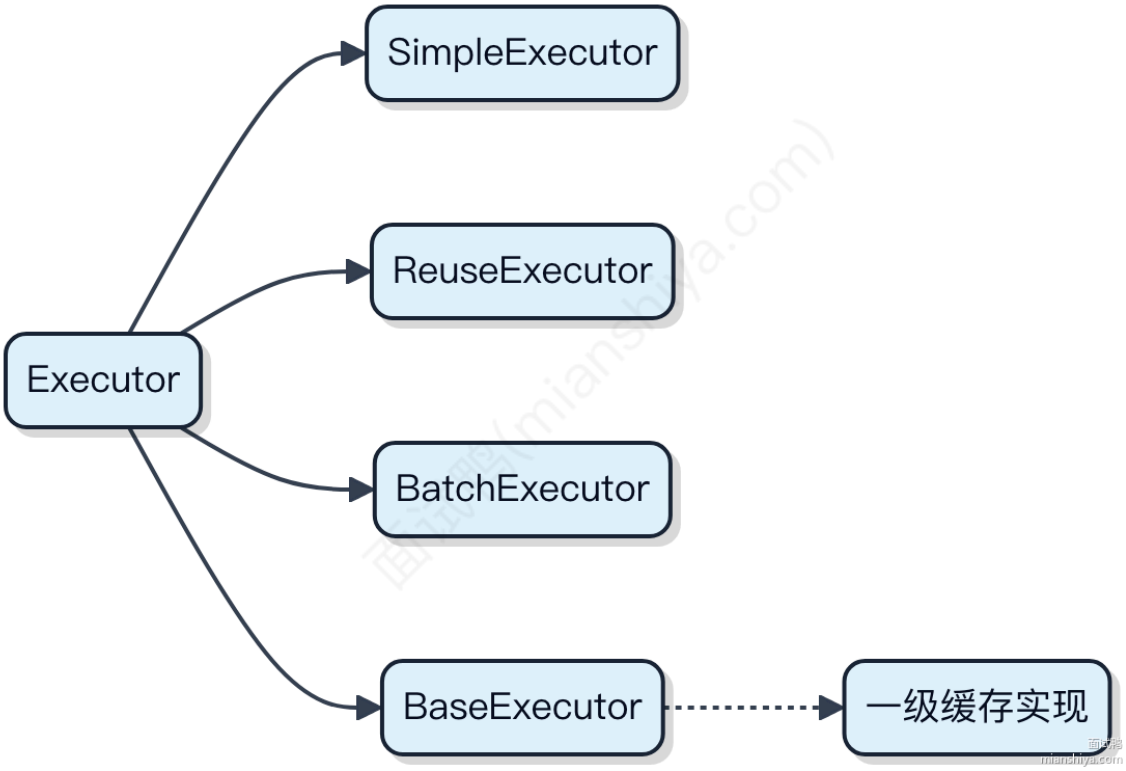
Mybatis 的执行器负责 SQL 语句的底层执行，控制着一级缓存、批量处理等核心行为。常见的有三种实现。

1）SimpleExecutor 是默认选择，每执行一次 update 或 select 就开启一个 Statement 对象，用完立刻关闭。简单直接，但频繁创建销毁 Statement，性能一般。

2）ReuseExecutor 会缓存 Statement 对象，相同 SQL 复用同一个 Statement 实例，避免重复创建。适合在一个 SqlSession 中反复执行相同 SQL 的场景，比如循环里查数据。不过它缓存的是 Statement，不是结果，一级缓存还得靠后面说的机制。

3）BatchExecutor 专为批量操作设计，只对 update（insert、delete）有效。它利用 JDBC 的 addBatch 和 executeBatch 来批量提交，能显著减少数据库通信次数。比如你一次性插 1000 条，用这个执行器可能只发 10 次请求而不是 1000 次，但要注意 commit 时才真正刷出去。

这三种执行器都继承自 BaseExecutor，一级缓存的实现就在父类里。缓存是 HashMap 结构，key 是 SQL、参数、环境等组成的 MappedStatement，value 是查询结果。每次查询先看缓存有没有，没有再走数据库。



选哪个执行器看场景：普通操作用 Simple，高频重复 SQL 用 Reuse，批量写入上 Batch。配置通过 `defaultExecutorType` 设置，也可以在 openSession 时指定。

MyBatis 如何实现数据库类型和 Java 类型的转换的？

MyBatis 通过 **TypeHandler** 完成数据库字段类型和 Java 类型之间的转换。每当执行 SQL 时，参数设置和结果映射阶段都会触发类型转换，框架根据目标类型自动选择合适的处理器。

1）每个 TypeHandler 负责一对类型的相互转换，比如 `Integer` ↔ `INT` 、 `LocalDateTime` ↔ `TIMESTAMP` 。MyBatis 内置了常见类型的处理器，像 `IntegerTypeHandler` 、 `StringTypeHandler` 等，覆盖基本数据类型和包装类。

2）自定义类型处理需要实现 `TypeHandler<T>` 接口，重写 `setParameter` 和 `getResult` 方法。例如枚举类保存到数据库时，可以用 `@EnumValue` 注解配合自定义处理器，把枚举转为 code 值存储。

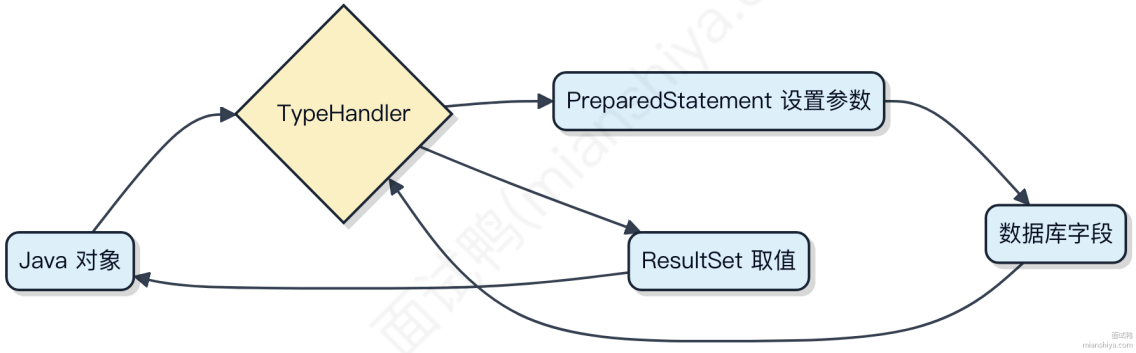
3）注册方式支持全局配置和注解两种。在 `mybatis-config.xml` 中注册后，MyBatis 会自动在参数映射和结果集解析时调用对应处理器，整个过程对开发者透明。

代码示例：

```
@MappedJdbcTypes(JdbcType.VARCHAR)
public class JsonTypeHandler implements TypeHandler<Object> {
    public void setParameter(PreparedStatement ps, int i, Object obj, JdbcType type)
{
        ps.setString(i, JSON.toJSONString(obj));
    }
    public Object getResult(ResultSet rs, String col) {
```

```
        return JSON.parseObject(rs.getString(col), Object.class);
    }
}
```

这种机制让 MyBatis 能灵活支持复杂对象，比如直接将 JSON 字段映射为 Java 对象，常用于处理 MySQL 的 JSON 类型，避免手动序列化反序列化。



能详细说说 MyBatis 的执行流程吗？

MyBatis 的执行流程，从发起一次 SQL 查询说起。你调用 Mapper 接口方法那一刻，其实背后走了一整套拦截、解析、执行的机制。

1）首先，Mapper 接口并不是实现类，而是通过动态代理生成的代理对象。每次你调 `userMapper.selectById(1)` ，代理就会根据方法名和参数，找到对应的 SQL 语句。

2）接下来进入 SqlSession，它负责统筹全局。SqlSession 会拿着MappedStatement（封装了SQL和配置）去执行查询。如果开启了二级缓存，先查二级缓存；没有就往下走。

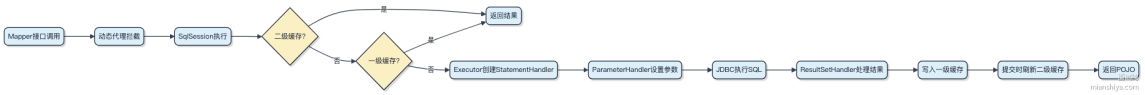
3）然后到 **Executor**，这是执行引擎的核心。它先查一级缓存（基于 SqlSession 的本地缓存），命中直接返回。没命中就创建 StatementHandler。

4）StatementHandler 负责处理 JDBC 的 Statement，做 SQL 拼接、参数映射。ParameterHandler 把你的入参按规则塞进预编译 SQL 里，比如 `?` 替换成实际值。

5）接着交给 JDBC 执行，ResultSet 返回后，ResultSetHandler 开始工作。它通过反射和类型处理器 TypeHandler，把数据库字段映射成 Java 对象。

6）结果出来后，写入一级缓存，如果有二级缓存且配置了，则在事务提交时刷入二级缓存。

整个过程链路清晰，组件各司其职。比如一级缓存默认开启，作用域是同一个 SqlSession；二级缓存需要手动开启，跨 SqlSession 共享，但要注意并发问题。



什么是 MyBatis-Plus？它有什么作用？它和 MyBatis 有哪些区别？

MyBatis-Plus 是在 MyBatis 基础上封装的增强工具，目标是简化开发、减少模板代码。你用过 MyBatis 就知道，增删改查写 XML 太重复，MyBatis-Plus 直接帮你把单表操作全包了，基本不用写 SQL。

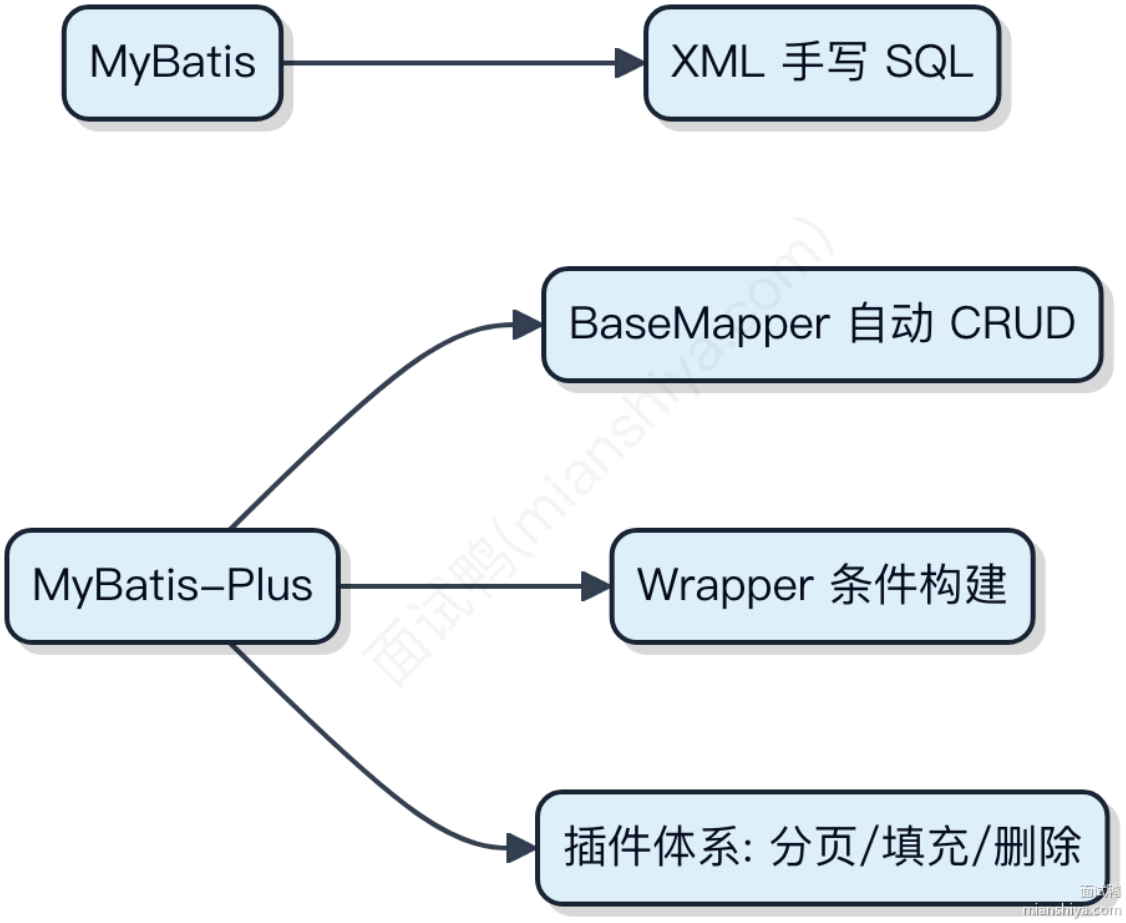
它最核心的功能是 **自动 CRUD** 和 **条件构造器**。比如实体类配上注解，Mapper 继承 BaseMapper，立刻拥有 insert、deleteById、selectList 等方法，连分页都能自动处理。

1）不需要再为每个单表写 XML 映射，90% 的简单操作直接调用接口就行
2）内置 QueryWrapper 构建查询条件，拼接 WHERE、LIKE、IN 都是链式调用，避免手写字符串 SQL
3）支持逻辑删除、自动填充创建时间/更新时间等常见需求，配置一下就能用

```
QueryWrapper<User> qw = new QueryWrapper<>();
qw.eq("status", 1).like("name", "张");
userMapper.selectList(qw);
```

和 MyBatis 的区别其实就两点：
一是开发效率，MyBatis-Plus 把通用操作抽成公共实现，你只专注业务复杂 SQL；
二是功能扩展，像分页插件 PageHelper 要配合使用，而 MP 内置分页、SQL 性能分析、租户插件等。

但复杂多表关联、存储过程还是得回退到 XML 或原生方式，毕竟它只是增强，不是替代。如果你项目里大量单表操作，用 MP 能省下不少脏活累活。



> *本资源来自面试鸭：https://www.mianshiya.com*

> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*

面试鸭(mianshiya.com)

面试鸭(mianshiya.com)